Cmpe 492 - Presentation 1

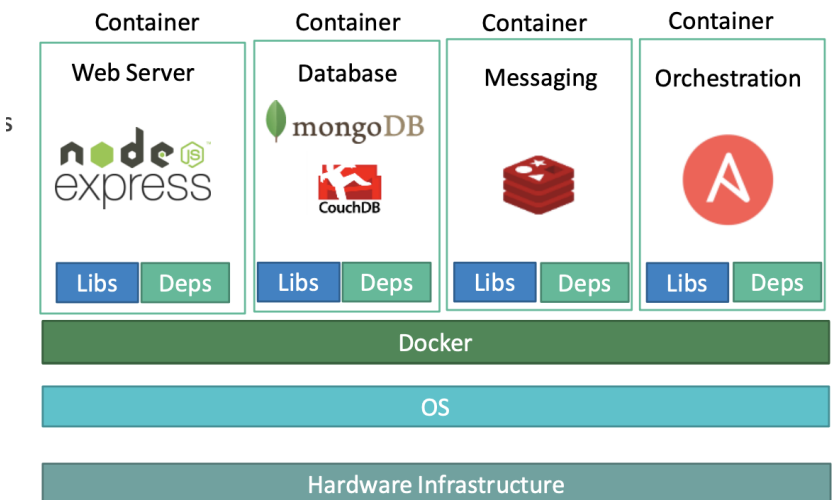
Mustafa Ocak - Halil İbrahim Kasapoğlu
Instructor: Atay Ozgovde

Bogazici University

Why Containers?

Managing modern applications without containers leads to:

- **OS Compatibility:** Struggles to align services with the host OS.
- **Dependency Issues:** Conflicts between libraries and dependencies across services.
- **Service Upgrades:** Changing a service requires re-testing the entire stack.
- **Environment Setup:** Setting up new environments is complex and slow.
- **Inconsistent Environments:** Differences between dev, test, and prod cause issues.



What is Docker?

Docker solves these problems by:

- **Packaging Apps:** Containers bundle code, libraries, and dependencies.
- **Consistency:** Containers run the same in any environment (dev, test, prod).
- **Isolation:** Containers are isolated, ensuring minimal interference between apps.
- **Efficiency:** Containers are lightweight and fast to deploy, scale, and manage.

Docker Architecture

Docker has four main components:

- **Docker Client:** Sends commands to the Docker Daemon.
- **Docker Daemon:** Manages Docker objects like images, containers, and networks.
- **Docker Registry:** Stores and distributes Docker images (e.g., Docker Hub).
- **Docker Containers:** Running instances of Docker images.

Docker Images

- **What are they?** Docker images are blueprints for containers.
- **What do they contain?** All necessary code, libraries, and configuration files.
- **Reusable:** Share and distribute via Docker Hub or private registries.

Docker Containers

- **What are they?** Containers are running instances of Docker images.
- **Isolated Environments:** Containers provide isolated environments for apps, avoiding interference.
- **Portability:** They can run consistently across any OS or platform.
- **Fast Deployment:** Containers are lightweight and quick to start or stop.

Dockerfile

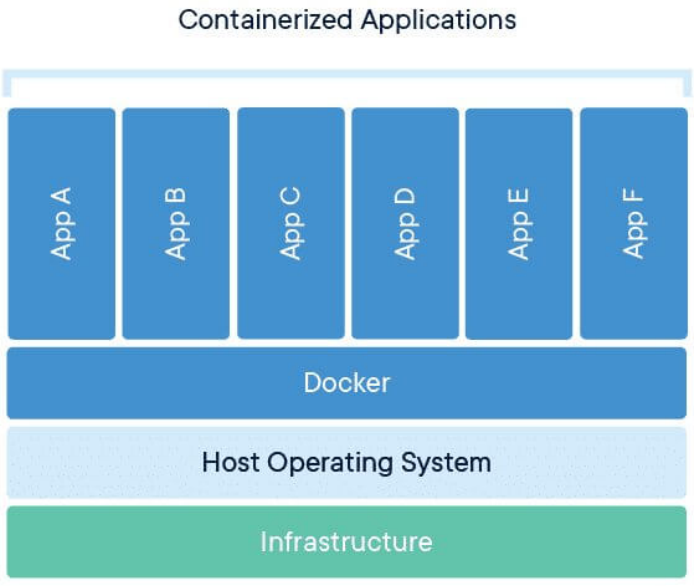
- **Purpose:** A Dockerfile automates the creation of Docker images.
- **Instructions:** Contains instructions to assemble the image (e.g., base image, commands, etc.).
- **Customizable:** You define what goes into the container.
- **Repeatable Builds:** Use the same Dockerfile to create identical containers every time.

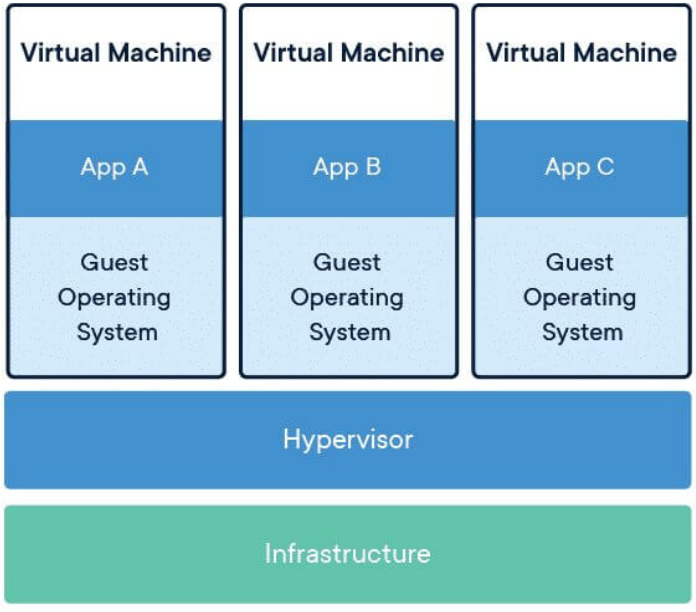
Docker Compose

- **What is it?** Tool to define and run multi-container Docker applications.
- **YAML File:** Define services, networks, and volumes in a single YAML file.
- **Command:** Start all services with one command: `docker-compose up`.
- **Example Use Case:** Running a web app with a database and cache.

Containers vs Virtual Machines

- **Virtual Machines:**
  - Full OS with dedicated kernel.
  - Heavier, slower to boot.
  - Requires more resources (CPU, memory).
  - Isolated environments but more overhead.
- **Containers:**
  - Share the host OS kernel.
  - Lightweight and faster to start.
  - Requires fewer resources.
  - Better suited for microservices and rapid scaling.

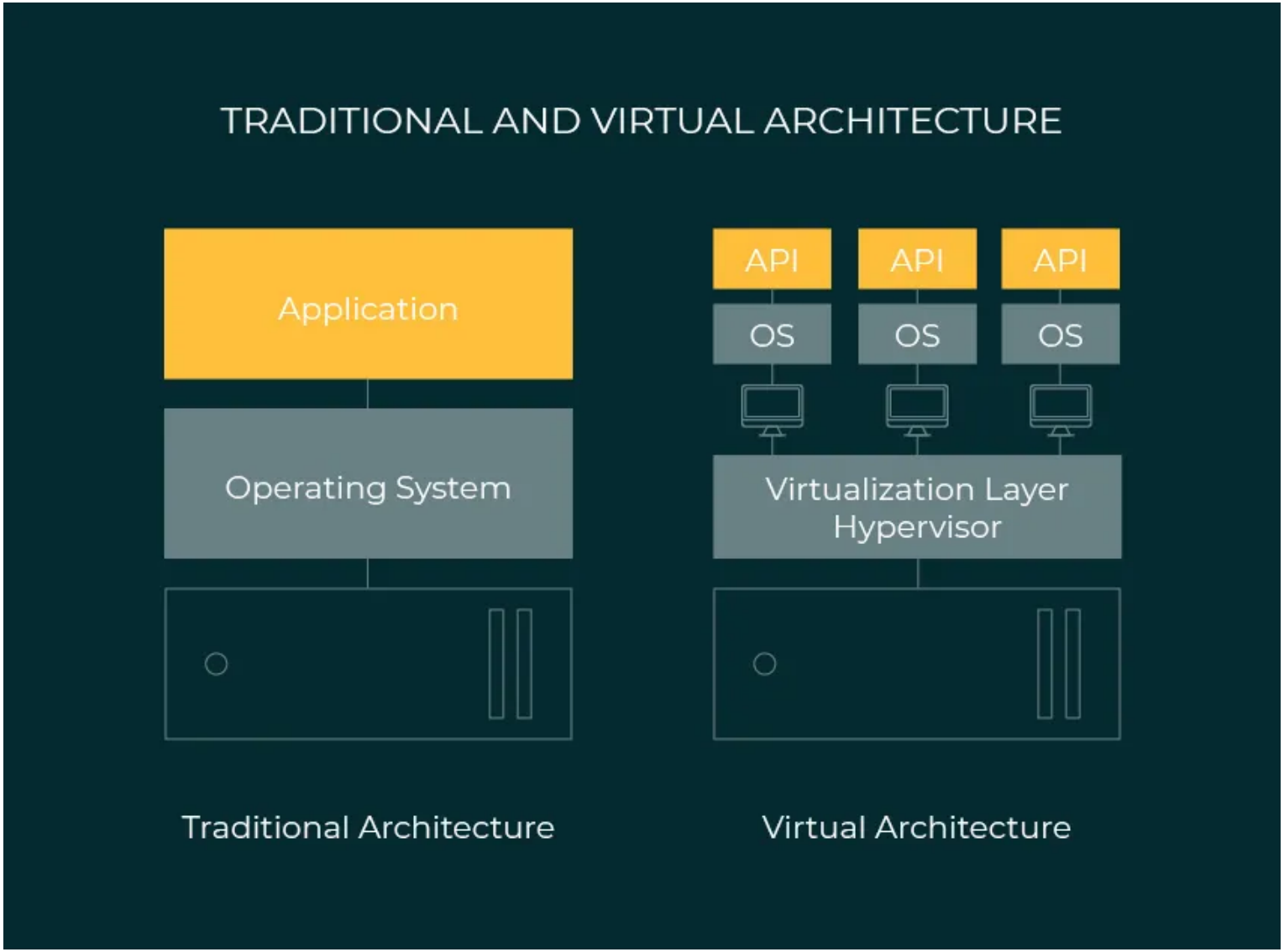




What is Virtualization?

- **Virtualization:**
  - The process of creating virtual versions of physical components, like servers, storage, or networks.
  - Multiple virtual machines (VMs) can run on a single physical machine, each with its own OS and applications.
- **Efficient Resource Management:**
  - Virtualization allows better utilization of hardware resources by partitioning them among multiple VMs.
  - It provides flexibility, scalability, and reduces hardware costs.
  - Cloud providers use virtualization to offer infrastructure as a service (IaaS).

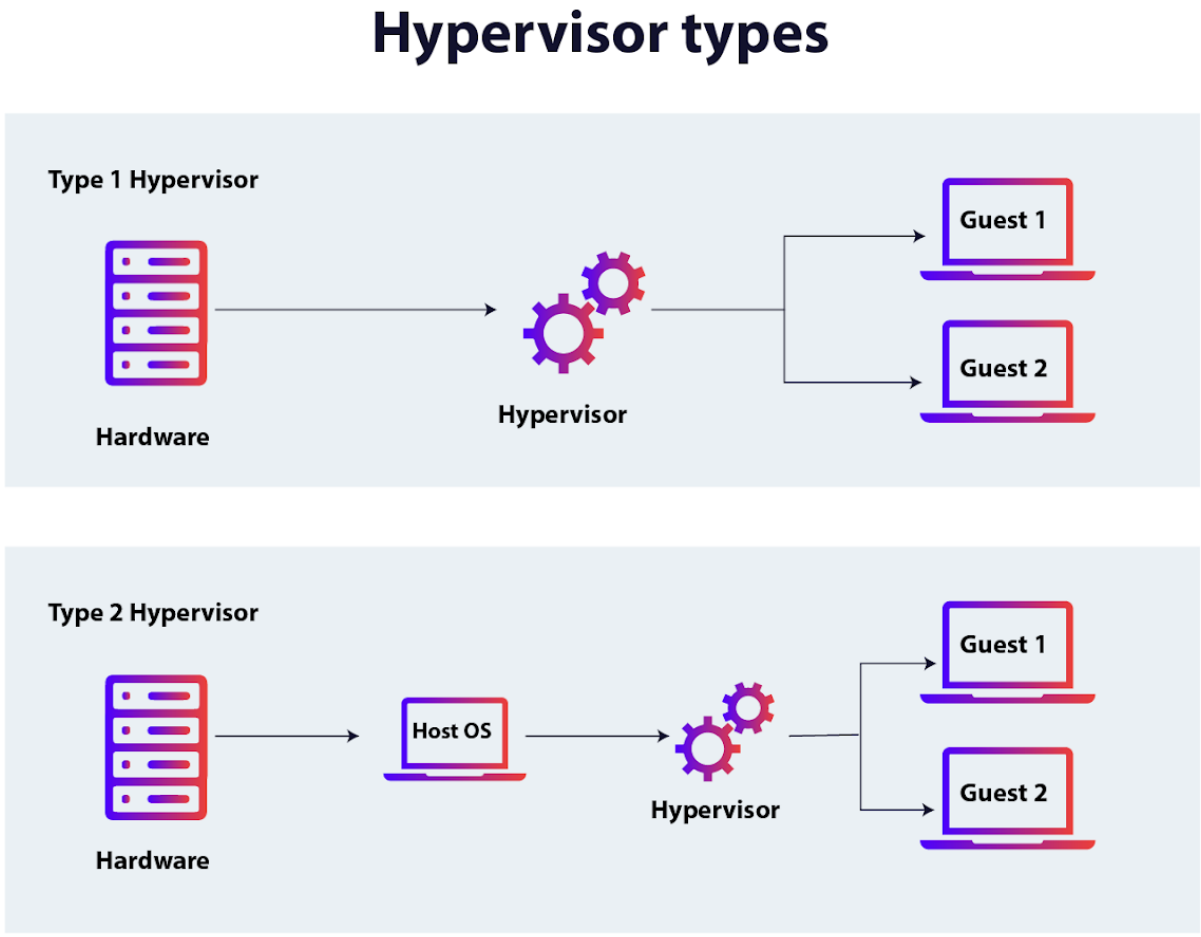
Traditional Architecture vs Virtual Architecture



How Do Hypervisors Work?

- **Hypervisors:** Software that creates and runs virtual machines (VMs).
- The hypervisor manages the resources of the physical machine and allocates them to virtual machines, allowing multiple VMs to run simultaneously on one physical host.
- **Two Types of Hypervisors:**
  - **Type 1 (Bare Metal):**
    - Runs directly on the hardware of the host machine.
    - Higher performance and efficiency since it bypasses the host OS.
    - Common in enterprise-level virtualization (e.g., VMware ESXi, Microsoft Hyper-V).
  - **Type 2 (Hosted):**
    - Runs on top of a host operating system.
    - Less efficient due to the overhead of the host OS.
    - More suitable for personal use or development (e.g., VirtualBox, VMware Workstation).

Hypervisor Types



Virtual Machines vs Physical Servers

- **Advantages of Virtual Machines:**
  - **Resource Efficiency:** Multiple VMs can run on a single physical server, optimizing resource usage.
  - **Isolation:** Each VM is isolated, minimizing risks of conflicts between applications.
  - **Scalability:** Easier to scale up or down by adding/removing VMs.
  - **Flexibility:** VMs can run different operating systems on the same physical hardware.
  - **Disaster Recovery:** Easy to create backups and snapshots of VMs for recovery.
- **Disadvantages of Virtual Machines:**
  - **Overhead:** VMs consume more resources than containers due to running full operating systems.
  - **Performance:** VMs may have slightly lower performance compared to running directly on physical hardware.
  - **Complexity:** Managing virtual infrastructure can be complex and require specialized tools.
  - **Licensing Costs:** Software licenses for hypervisors and OSes can increase operational costs.

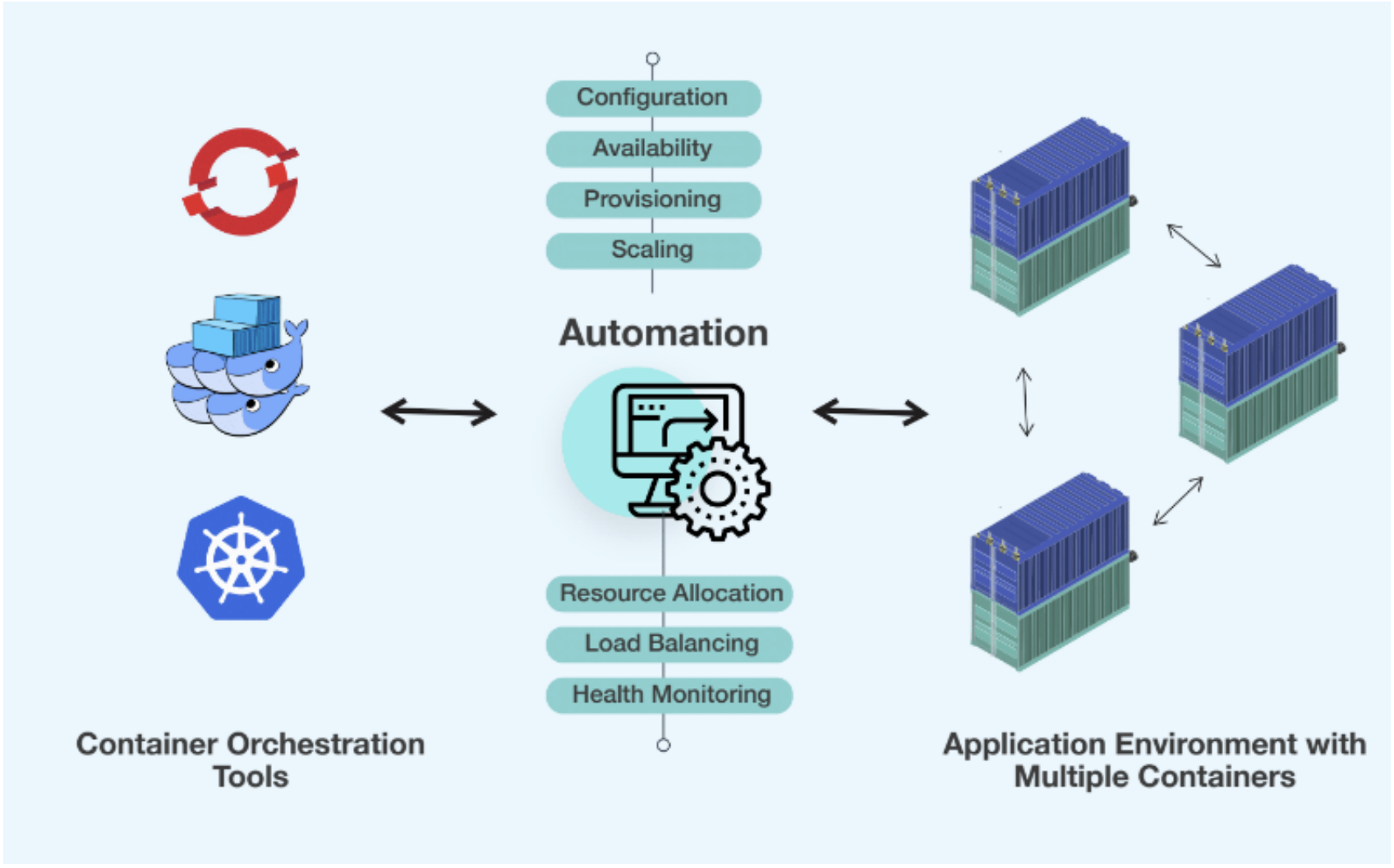
What is Container Orchestration?

- **Container Orchestration:**
  - The automated management of containers in production environments.
  - It handles tasks such as deployment, scaling, and networking of containers.
  - Orchestration tools ensure that containerized applications are running smoothly across multiple hosts.
  - Helps in managing complex, distributed container-based architectures.

Why Do We Need Container Orchestration?

- **Why Container Orchestration is Needed:**
  - **Scaling:** Automatically adjusts the number of running containers based on demand.
  - **Load Balancing:** Distributes traffic evenly across containers to ensure high availability.
  - **Self-Healing:** Detects failed containers and replaces them to maintain system reliability.
  - **Multi-Host Management:** Coordinates containers across multiple servers, ensuring efficient resource use.
  - **Service Discovery:** Automatically manages communication between containers.

Container Orchestration Diagram



Advantages of Container Orchestration

- **High Availability:**
  - Hardware failures do not bring down the application.
  - Multiple instances of the application run on different nodes, ensuring availability.
- **Load Balancing:**
  - Traffic is automatically balanced across all running containers.
  - Ensures efficient use of resources and maintains application performance.

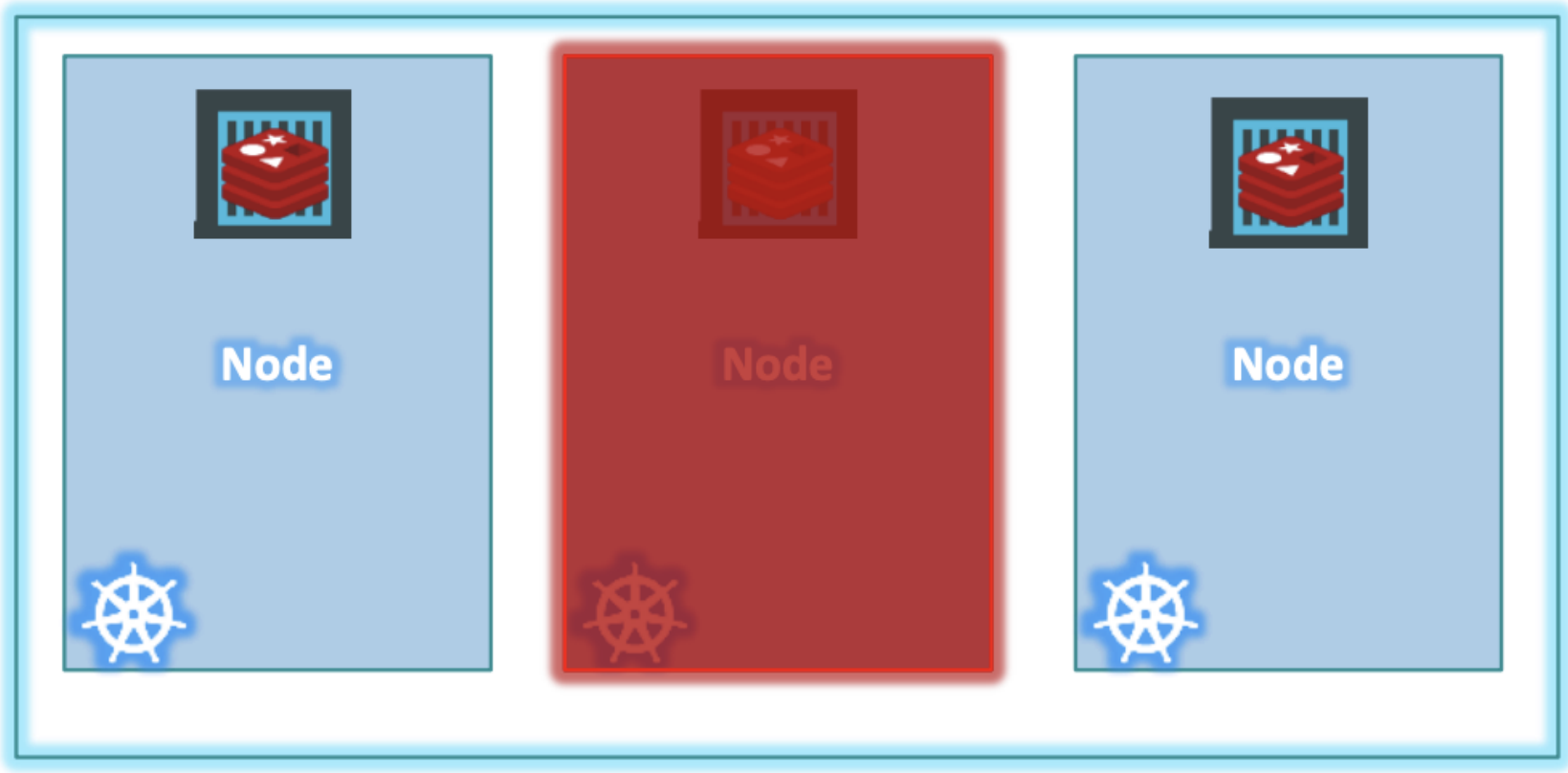
Advantages of Container Orchestration

- **Scaling:**
  - Seamlessly deploy more instances when demand increases, scaling at the service level.
  - Easily scale up or down the number of nodes without taking down the application.
- **Declarative Configuration:**
  - Use declarative object configuration files to manage these tasks with ease.
  - No manual intervention is needed to manage the scaling and load balancing.

What is a Kubernetes Node?

- **Kubernetes Node:**
  - A machine that runs containerized workloads as part of a Kubernetes cluster.
  - A node can be a physical machine or a virtual machine.
  - Nodes can be hosted on-premises or in the cloud.
- **Kubernetes Cluster:**
  - A set of nodes grouped together to run containerized workloads.
  - Ensures high availability—if one node fails, the application remains accessible from other nodes.
  - Helps distribute the load across multiple nodes, enhancing performance and scalability.

Node and Cluster



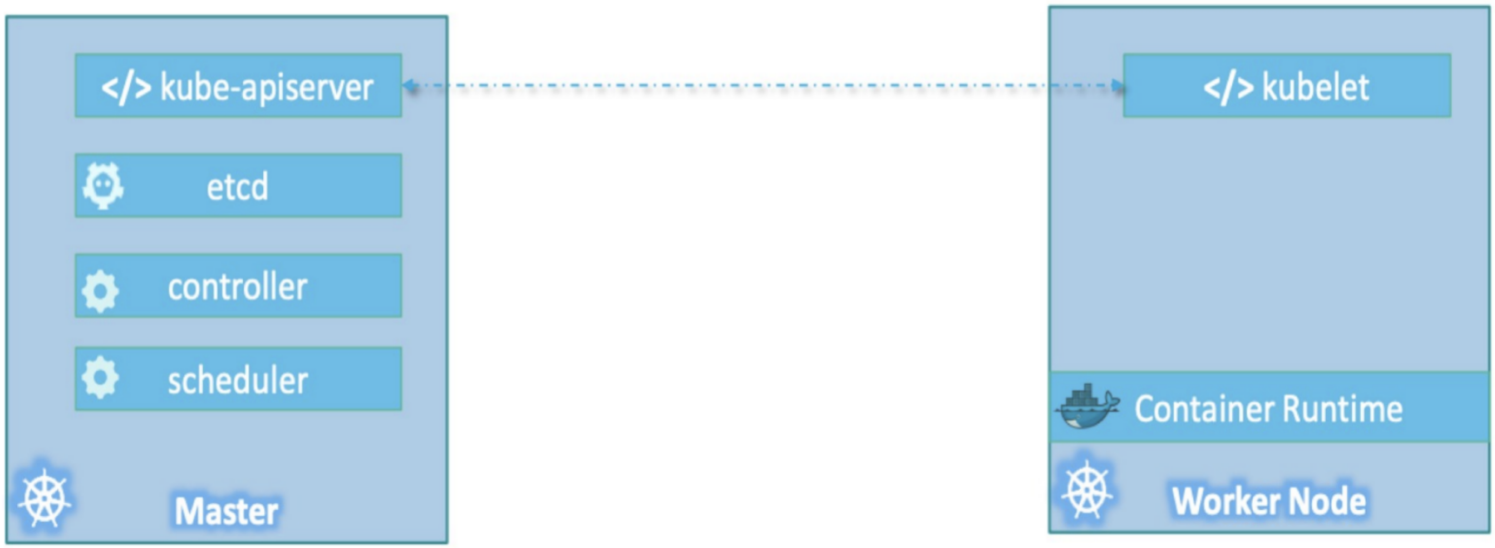
Master and Worker Nodes in Kubernetes

- **Master Node:**
  - Runs control plane components such as **kube-apiserver**, **etcd**, **controller manager**, and **scheduler**.
  - Manages the entire cluster, orchestrating workloads across worker nodes.
- **Worker Node:**
  - Hosts the actual containerized workloads, such as Docker containers.
  - Each worker node runs the **kubelet**, which communicates with the master to execute tasks.

Container Runtime

- **Container Runtime:**
  - The software that runs containers on the worker node.
  - Examples: **Docker**, **Rocket**, **CRIO**, etc.

Master and Worker Nodes Diagram



Role of Container Orchestration in Cloud Infrastructure

- **Automated Management:**
  - Simplifies the deployment, scaling, and management of containerized applications.
  - Automates processes like starting, stopping, and updating containers.
- **High Availability:**
  - Ensures that applications remain available even in case of node failures.
  - Provides mechanisms for failover and recovery, maintaining service uptime.
- **Scalability:**
  - Automatically scales applications by adding or removing containers based on demand.
  - Enables seamless scaling across multiple nodes or regions.

Role of Container Orchestration in Cloud Infrastructure (Cont'd)

- **Resource Optimization:**
  - Efficiently allocates CPU, memory, and other resources across containers.
  - Reduces resource wastage and optimizes hardware usage.
- **Service Discovery and Networking:**
  - Manages communication between containers in a seamless, automated way.
  - Ensures that services find each other even as containers are started, stopped, or moved.
- **Security:**
  - Implements security policies and access controls for containerized workloads.
  - Isolates containers to minimize security risks.

Kubernetes Components Overview

- When installing Kubernetes, the following core components are installed:
  - **API Server**
  - **ETCD Service**
  - **Kubelet Service**
  - **Container Runtime**
  - **Controllers**
  - **Schedulers**

API Server

- **API Server:**
  - Acts as the front-end for the Kubernetes control plane.
  - Users, management devices, and command-line interfaces interact with the Kubernetes cluster through the API server.
  - The main entry point for all administrative tasks, making the node a master.
  - Receives commands and distributes them across worker nodes.

ETCD Key Store

- **ETCD:**
  - A distributed and reliable key-value store used to store all data related to managing the cluster.
  - Stores information about all nodes, masters, and workloads across the cluster in a distributed manner.
  - Responsible for implementing locks within the cluster to avoid conflicts between multiple masters.

Scheduler

- **Scheduler:**
  - Responsible for distributing workloads (containers) across multiple nodes.
  - Assigns newly created containers to nodes based on available resources and constraints.

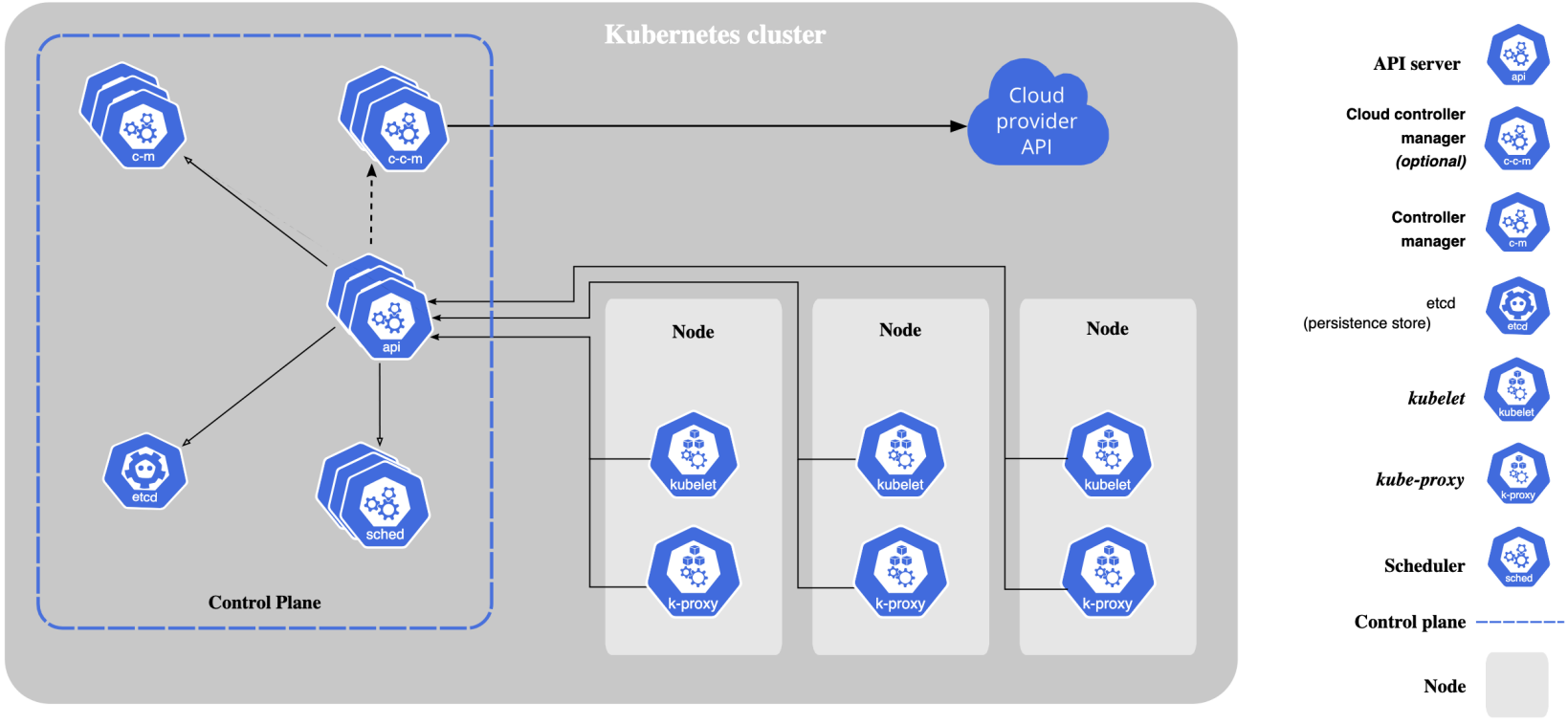
Controllers

- **Controllers:**
  - The "brain" behind Kubernetes orchestration.
  - Responsible for monitoring the state of nodes, containers, and endpoints.
  - Takes action when failures occur, such as bringing up new containers when existing ones go down.

Kubelet

- **Kubelet:**
  - The agent running on worker nodes that communicates with the master node.
  - Interact with both the container and node.
  - Responsible for reporting node health and carrying out tasks assigned by the master.
  - Responsible for ensuring that the containers are running as expected on each node.
  - Continuously communicates with the master to execute container-related tasks and report node status.

Kubernetes Components Diagram



What is kubectl?

- **kubectl:**
  - The command-line tool for interacting with a Kubernetes cluster.
  - Provides commands to manage and control Kubernetes resources like nodes, pods, services, and deployments.
  - Sends requests to the Kubernetes API server to execute tasks.

What is a Pod?

- **Pod:**
  - The smallest, most basic deployable unit in Kubernetes.
  - A Pod represents one or more containers that share storage, networking, and a specification on how to run.
  - Containers in a Pod are tightly coupled, sharing the same network namespace and IP address.
- **Usage:**
  - Typically used to run a single instance of a microservice.
  - Pods can contain multiple containers (e.g., sidecar containers for logging, proxies). The two containers can also communicate with each other directly by referring to each other as 'localhost' since they share the same network namespace. Plus they can easily share the same storage space as well.
  - But in general we have one to one relationship between container and pod.

Common Pod Commands

- **Common ‘kubectl‘ Commands for Pods:**
  - `kubectl get pods` – List all running Pods in the cluster.
  - `kubectl describe pod <pod-name>` – Get detailed information about a specific Pod.
  - `kubectl logs <pod-name>` – View the logs of a container within a Pod.
  - `kubectl exec -it <pod-name> - <command>` – Execute a command in a running container within a Pod.
  - `kubectl delete pod <pod-name>` – Delete a Pod.

Kubernetes YAML Files Overview

- Kubernetes uses YAML files to create objects like Pods, ReplicaSets, Deployments, and Services.
- Every Kubernetes YAML file follows a similar structure with 4 required fields:
  - **apiVersion**
  - **kind**
  - **metadata**
  - **spec**
- These fields must be present in every Kubernetes definition file.

apiVersion and kind

- **apiVersion:** Specifies the version of the Kubernetes API being used.
- **kind:** Refers to the type of Kubernetes object.
  - For Pods, set it as `Pod`.
  - Other possible values include `ReplicaSet`, `Deployment`, or `Service`.

metadata

- **metadata:** Contains data about the object like its name and labels.
  - **name:** Specifies the name of the object, e.g., `myapp-pod`.
  - **labels:** A dictionary of key-value pairs used to categorize and identify objects (e.g., `app: myapp`).
- Labels can be useful for filtering and managing objects later, especially when dealing with large numbers of Pods.
- Ensure proper indentation for all elements under `metadata`.

What is a ReplicaSet?

- **ReplicaSet:**
  - Ensures that a specified number of identical Pods are running at all times.
  - Automatically creates or deletes Pods to maintain the desired number of replicas.
  - Provides high availability by ensuring that application instances are always running.
- **Usage:**
  - Commonly used to maintain the desired number of Pods in a deployment.
  - If a Pod fails, the ReplicaSet automatically replaces it.

What is a Deployment?

- **Deployment:**
  - Manages the deployment and scaling of ReplicaSets.
  - Ensures a desired state for the application, including the number of Pods running.
  - Supports updates and rollbacks of the application seamlessly.
  - Ensures that the correct number of Pods are running and replaces any failed Pods.

Why Use Kubernetes Deployments?

- **Multiple Instances:**
  - In production, you need multiple instances of an application to ensure high availability and load distribution.
- **Seamless Upgrades:**
  - When new application versions are available, you want to upgrade Docker instances seamlessly without downtime.
- **Rolling Updates:**
  - Updates should not occur all at once. Kubernetes Deployments allow for **rolling updates**, updating instances one by one to minimize impact on users.
- **Rollback:**
  - If an update causes unexpected issues, you can easily **rollback** to a previous version.
- **Pausing and Resuming Changes:**
  - You can **pause** your environment to make multiple changes (like scaling, updating resources), then **resume** and apply all changes together.

Role of Deployments in Kubernetes

- A Deployment manages Pods and ReplicaSets to ensure the desired state of your application.
- **Capabilities of a Deployment:**
  - **Rolling Updates:** Deploy new versions of your application incrementally.
  - **Rollback:** Revert to a previous version if the current one has issues.
  - **Pause/Resume:** Temporarily halt updates to make multiple changes at once and then resume the deployment.

Example Deployment

Definition

```
> kubectl create -f deployment-definition.yml
deployment "myapp-deployment" created
```

```
> kubectl get deployments
NAME                DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
myapp-deployment    3         3         3            3           21s
```

```
> kubectl get replicaset
NAME                          DESIRED   CURRENT   READY   AGE
myapp-deployment-6795844b58   3         3         3       2m
```

```
> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
myapp-deployment-6795844b58-5rbjl   1/1     Running   0          2m
myapp-deployment-6795844b58-h4w55   1/1     Running   0          2m
myapp-deployment-6795844b58-lfjhv   1/1     Running   0          2m
```

deployment-definition.yml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-deployment
  labels:
      app: myapp
      type: front-end
spec:
  template:
    metadata:
      name: myapp-pod
      labels:
         app: myapp
         type: front-end
    spec:
      containers:
      - name: nginx-container
        image: nginx

  replicas: 3
  selector:
     matchLabels:
        type: front-end
```

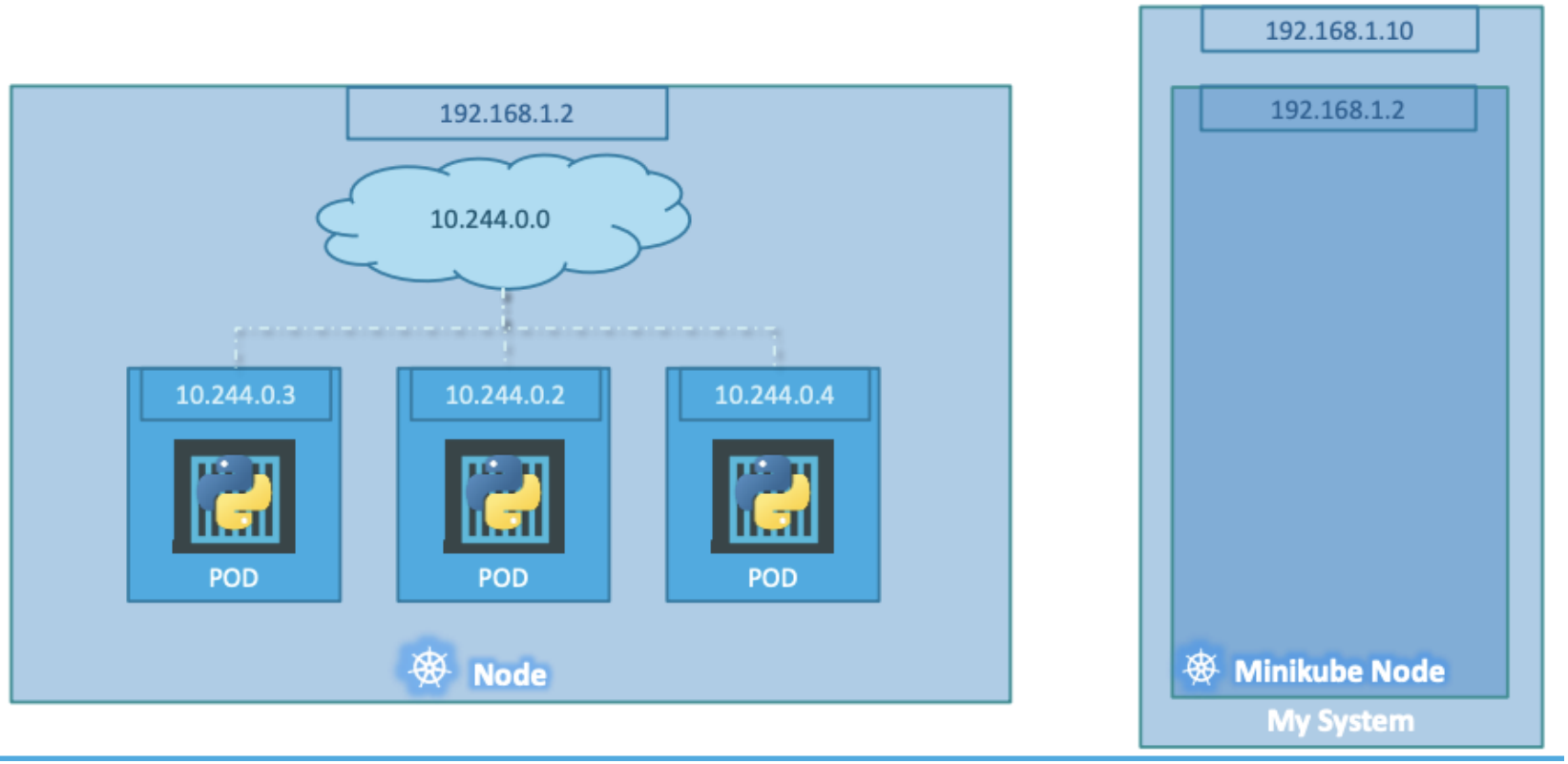
How to Create a Deployment

- **Create a Deployment Definition File:**
  - The deployment definition file is similar to the ReplicaSet definition file, except for the `kind` field, which is set to `Deployment`.
  - The YAML file contains:
    - **apiVersion:** Set to `apps/v1`.
    - **metadata:** Includes name and labels.
    - **spec:** Contains template, replicas, and selector fields, with a Pod definition inside the template.
- **Steps to Create the Deployment:**
  - Run `kubectl create -f deployment-definition.yml` to create the deployment.
  - Run `kubectl get deployments` to verify the newly created deployment.
  - The deployment automatically creates a ReplicaSet, which in turn creates Pods.

Introduction to Kubernetes Networking

- **Node IP Address:**
  - Each node in a Kubernetes cluster has its own IP address.
  - For example, a single-node cluster may have an IP address of `192.168.1.2`.
  - This IP is used to SSH into the node or interact with it.
- **Pod IP Address:**
  - Pods are attached to an internal network and they have their own IP addresses assigned.
  - In Kubernetes, IP addresses are assigned to Pods, not directly to containers.
  - Each Pod gets a unique internal IP address, e.g., `10.244.0.2`.
  - This internal IP allows Pods to communicate within the Kubernetes network.

Example Network of Node



Practical Work - VM Infrastructure

- Created a VM using UTM (uses QEMU) on a Silicon MacBook.
- Installed ARM Ubuntu Server 24.04.1 LTS on the VM.
- VM Specs: 1 CPU, 512MB RAM.
- Installed Apache web server on the VM.
- Configured the Apache server to serve custom content.
- Conducted load tests using Apache Benchmark (ab).

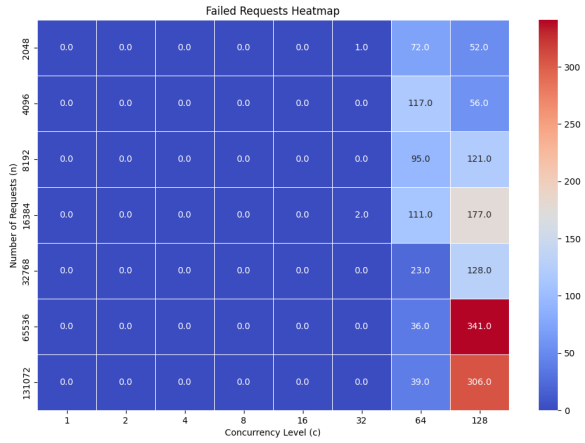
Practical Work - Kubernetes Infrastructure

- Installed Minikube on the local machine.
- Minikube Driver: Docker.
- Created a Dockerfile for Apache server image.
- Deployed Apache server on Kubernetes using Minikube.
- Conducted load tests on Kubernetes-deployed Apache server.
- Enabled Horizontal Pod Autoscaler (HPA) on Kubernetes.
- Conducted load tests with HPA enabled.

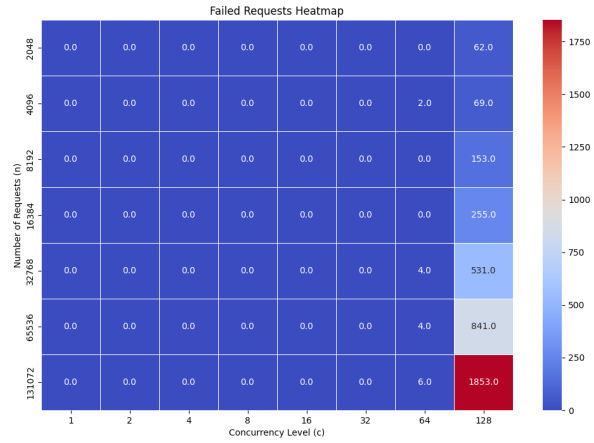
Practical Work - Load Testing

- Conducted load tests using Apache Benchmark (ab) tool.
- Automated the testing process using a bash script.
- Analyzed the test results using a Python script.
- Generated plots for Requests per second, Time per request, and Failed requests.
- Compared the performance of VM-based Apache server and Kubernetes-deployed Apache server.

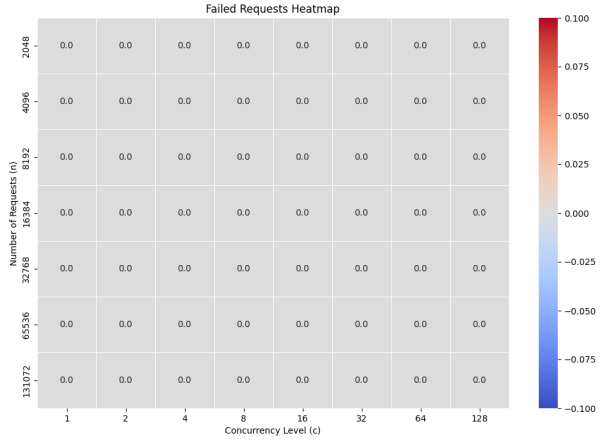
Failed Requests - VM vs Kubernetes vs Kubernetes with HPA



VM

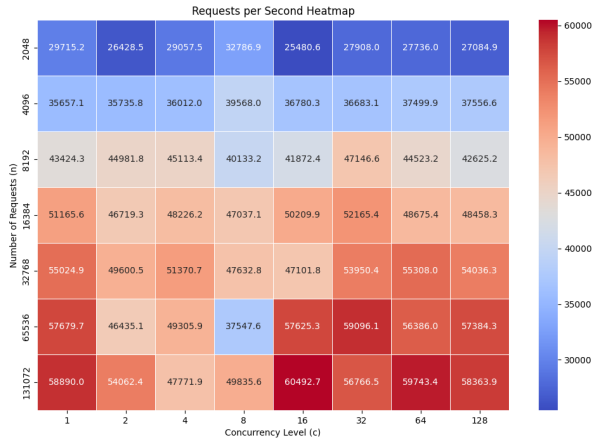


K8S

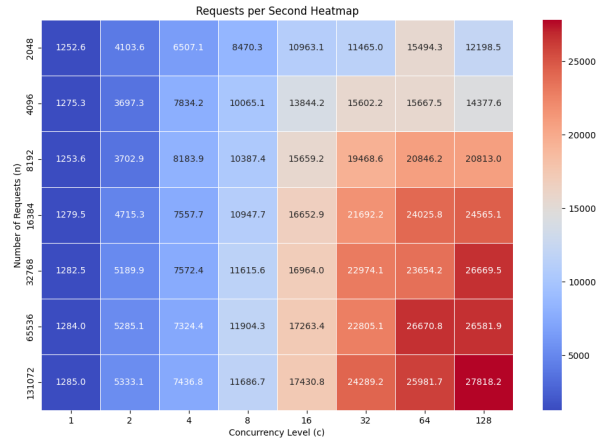


K8S HPA

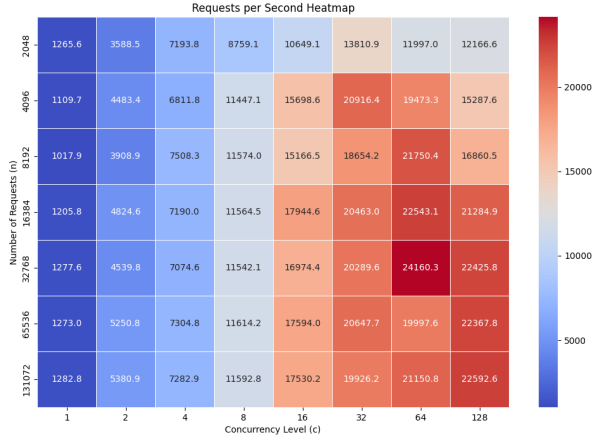
Requests per Second - VM vs Kubernetes vs Kubernetes with HPA



VM

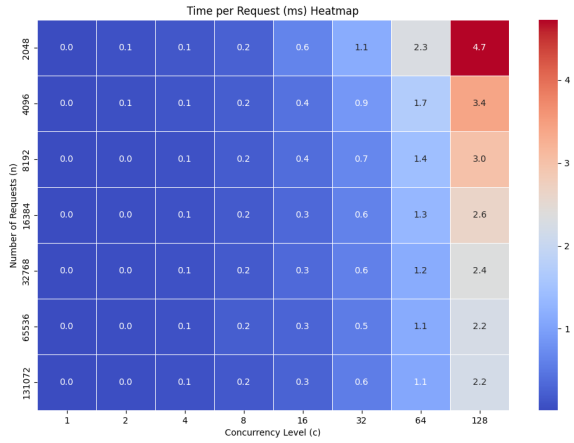


K8S

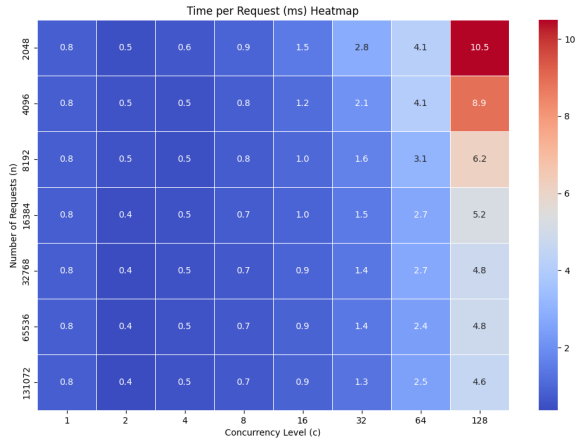


K8S HPA

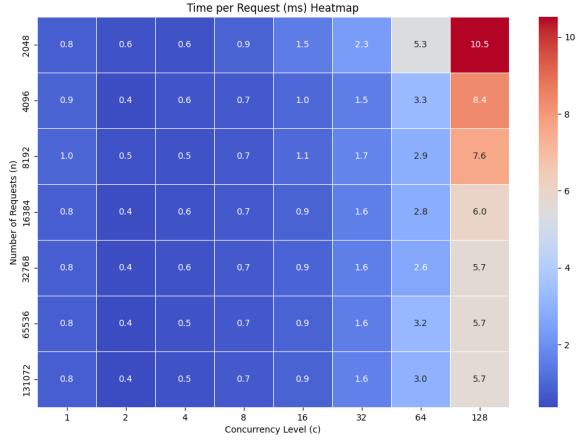
Time per Request - VM vs Kubernetes vs Kubernetes with HPA



VM

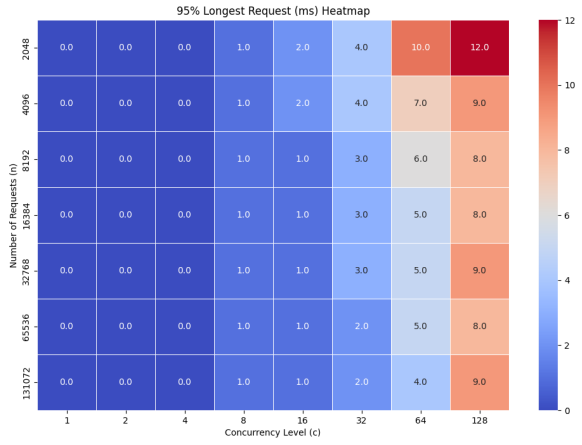


K8S

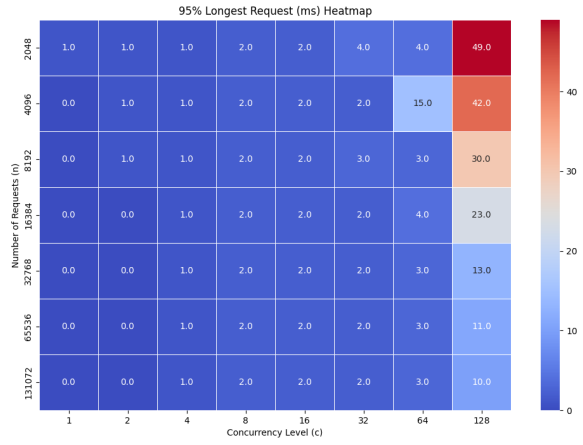


K8S HPA

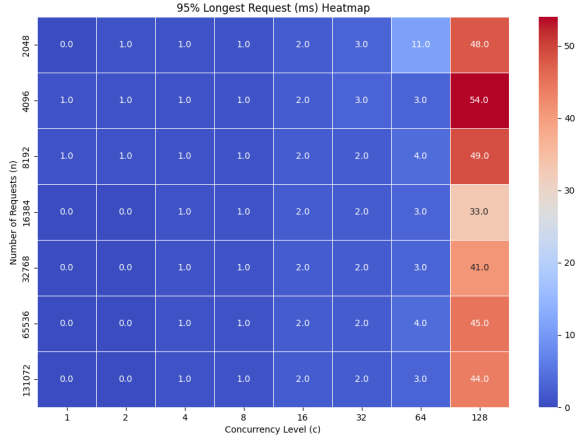
95% Longest Request - VM vs Kubernetes vs Kubernetes with HPA



VM

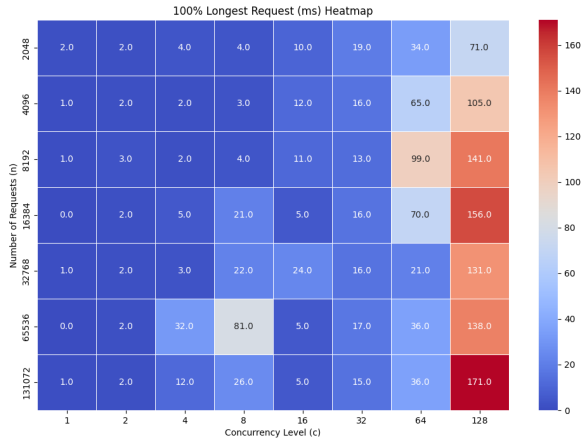


K8S

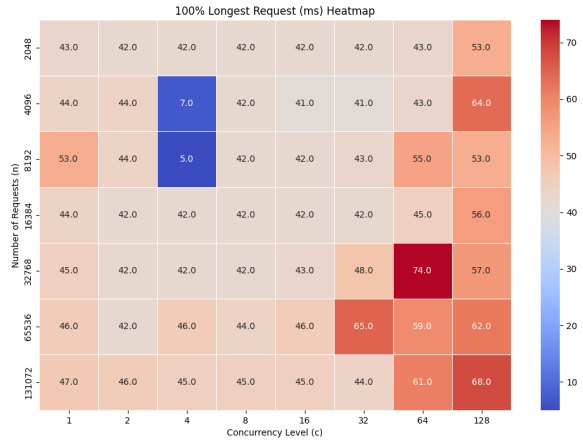


K8S HPA

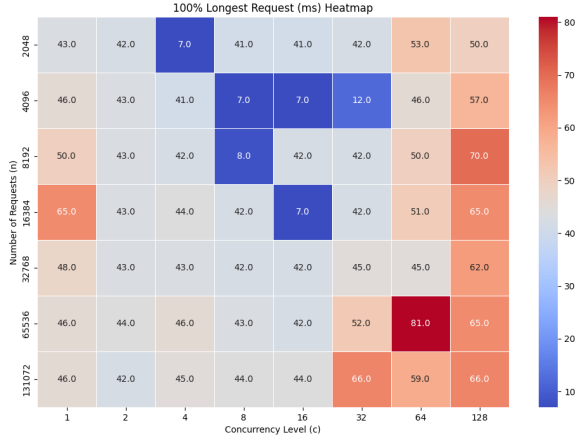
100% Longest Request - VM vs Kubernetes vs Kubernetes with HPA



VM



K8S



K8S HPA